Informe técnico — Simulador de gestión de memoria

Nov 2, 2026 · @Juan

Curso: Sistemas Operativos — Proyecto final, Opción 2: Gestión de memoria

Integrantes: Juan Miguel Villar Gómez · Roger Romero · Yeison Rojas

Docente: Sergio Lubo

Fecha: 3 de noviembre de 2026

1. Descripción del problema

Cuando el sistema operativo asigna a cada proceso una única sección contigua de memoria, la memoria libre termina partida en huecos de distintos tamaños. Al llegar una solicitud, el sistema debe decidir **qué hueco usar**, y esa decisión determina cuánta memoria queda inutilizable más adelante.

El problema tiene dos formas de desperdicio:

- **Fragmentación externa:** hay memoria libre suficiente para atender una solicitud, pero está repartida en huecos no contiguos y ninguno basta.
- **Fragmentación interna:** cuando la memoria se entrega en bloques de tamaño fijo, el proceso recibe más de lo que pidió y la diferencia queda sin uso dentro de su partición.

Las estrategias clásicas para elegir el hueco (primer ajuste, mejor ajuste y peor ajuste) prometen comportamientos distintos, pero su efecto real depende de la carga del sistema y del tamaño de la unidad de asignación. Este proyecto construye un simulador para medir esas diferencias con datos, en lugar de suponerlas.

2. Objetivo del proyecto

Objetivo general: implementar un simulador de asignación contigua de memoria con particionado dinámico que permita comparar, con resultados medidos y reproducibles, el comportamiento de tres estrategias de asignación de huecos.

Objetivos específicos:

1. Simular la asignación y liberación de memoria, incluyendo la división de huecos y la fusión de huecos adyacentes.
2. Representar en todo momento los espacios libres y ocupados, y verificar su consistencia después de cada operación.
3. Comparar primer ajuste, mejor ajuste y peor ajuste aplicando a las tres la misma secuencia de solicitudes.
4. Medir la fragmentación interna y externa, la utilización, la tasa de rechazo y el coste de búsqueda.
5. Evaluar tres niveles de carga y tres tamaños de unidad de asignación, y contrastar los resultados con lo que afirma la teoría.

3. Fundamentación del mecanismo seleccionado

El simulador implementa la **asignación contigua con particionado dinámico**, tal como se presenta en los apuntes de la asignatura. El sistema operativo mantiene una tabla que indica qué partes de la memoria están libres y cuáles ocupadas; al inicio toda la memoria es un único hueco.

Asignación. Al llegar una petición se busca un hueco suficientemente grande. Se asigna solo el espacio necesario y el resto queda como un hueco más pequeño.

Liberación. Cuando un proceso termina, su partición vuelve a ser un hueco y se funde con los huecos adyacentes, para no partir la memoria libre sin necesidad.

Fragmentación externa. Ocurre cuando hay suficiente espacio libre para satisfacer una petición, pero ese espacio no es contiguo. Las soluciones clásicas son la compactación, que solo es posible si la reubicación se hace en tiempo de ejecución, y los espacios de direcciones no contiguos, como la paginación.

Fragmentación interna. Registrar huecos de tamaño arbitrario no compensa, porque la información para registrar un hueco de pocos bytes supera su tamaño. Por eso se asigna en múltiplos de un bloque de tamaño fijo. Ejemplo de los apuntes: con bloques de 4 KiB, una petición de 9 KiB recibe 12 KiB y deja 3 KiB sin uso.

Regla del 50 %. Según los análisis estadísticos citados en los apuntes, con primer ajuste y N bloques asignados se pierden alrededor de 0,5·N bloques por fragmentación externa. El simulador mide esta relación directamente.

Por qué este mecanismo. Es la base sobre la que se entienden la paginación y la memoria virtual: muestra con claridad el problema que esas técnicas resuelven. Además, permite comparar estrategias bajo condiciones controladas.

4. Diseño de la solución

El simulador está escrito en C11, sin memoria dinámica (`malloc`): todas las estructuras son arreglos de tamaño fijo, lo que elimina las fugas de memoria. Se compila con `gcc -std=c11 -Wall -Wextra -pedantic -Werror`.

Representación de la memoria. La memoria es una **lista de bloques ordenada por dirección**. Cada bloque guarda su dirección de inicio, su tamaño, si está libre, el proceso dueño y el tamaño que ese proceso pidió. Entre todos cubren la memoria sin solaparse:

```
inicio:  0          12    17                67    72          92     100
         | libre 12 | P2 5 | libre 50         | P4 5 | libre 20  | P6 8 |
```

Se eligió frente al mapa de bits porque buscar un hueco recorre bloques y no bits, y porque al estar ordenada los vecinos físicos de un bloque son el anterior y el siguiente de la lista, lo que hace inmediata la fusión de huecos. Con P procesos puede haber como máximo 2P + 1 bloques; el arreglo se dimensiona con ese límite.

Separación entre núcleo y estrategia. Las tres estrategias comparten el mismo código de asignación y liberación; una estrategia solo **elige** el hueco y se pasa como función. Así, cualquier diferencia en los resultados se debe a la estrategia y no a errores distintos en tres implementaciones. El núcleo, además, rechaza un índice inválido devuelto por una estrategia en vez de corromper la memoria.

Módulos:

Archivo	Responsabilidad
`include/memoria.h`	Contrato: estructuras, funciones e invariantes
`src/memoria.c`	Asignar (redondeo a la unidad y división del hueco) y liberar (fusión con vecinos libres)
`src/estrategias.c`	Primer, mejor y peor ajuste
`src/invariantes.c`	Verificación de consistencia I1–I7

Archivo	Responsabilidad
`src/metricas.c`	Fragmentación, utilización, coste de búsqueda y promedios en el tiempo
`src/secuencia.c`	Lectura de archivos `A <pid> <KB>` / `L <pid>` y generador reproducible
`src/main.c`	Opciones, ejecución de las estrategias, tabla y CSV

Invariantes verificados después de cada operación. Si alguno falla, el programa se detiene en lugar de producir métricas falsas.

Id	Condición
I1	Los bloques son contiguos: cada uno empieza donde termina el anterior
I2	El primero empieza en 0 y la suma de tamaños es igual a la memoria total
I3	Todo tamaño es positivo y múltiplo de la unidad
I4	No hay dos huecos seguidos (la fusión siempre se hizo)
I5	Cada proceso aparece una sola vez
I6	En un bloque ocupado, lo asignado menos lo pedido es menor que una unidad
I7	Un hueco no tiene dueño; un bloque ocupado sí

Reproducibilidad. El generador aleatorio es propio (xorshift32) y no `rand()`, porque `rand()` produce secuencias distintas en Linux y en macOS. La misma semilla da los mismos resultados en cualquier computador.

5. Estrategias y configuraciones comparadas

Se comparan tres estrategias de asignación de huecos y tres tamaños de unidad de asignación.

Estrategia	Qué hueco elige	Cómo busca	Idea detrás
Primer ajuste (*first fit*)	El primero que sirve	Se detiene al encontrarlo	Rapidez
Mejor ajuste (*best fit*)	El más pequeño que sirve	Recorre toda la lista; solo corta antes si encuentra un hueco exacto	Dejar el sobrante más pequeño
Peor ajuste (*worst fit*)	El más grande	Recorre toda la lista	Que el sobrante siga siendo útil para otro proceso

Desempate: si dos huecos tienen el mismo tamaño, gana el de menor dirección.

Validación con el ejemplo de los apuntes. Con huecos de 15, 4, 20 y 13 KB y una petición de 12 KB, el simulador elige el hueco de 15 KB con primer ajuste (sobran 3), el de 13 KB con mejor ajuste (sobra 1) y el de 20 KB con peor ajuste (sobran 8). Coincide con lo indicado en los apuntes. La traza completa está en `resultados/traza_ejemplo_apuntes.txt`.

Unidad de asignación: 1, 4 y 16 KB. Una petición se redondea hacia arriba al múltiplo de la unidad; con 16 KB, un proceso que pide 40 KB recibe 48 KB.

6. Escenarios y métricas de prueba

Todos los escenarios usan 256 KB de memoria, procesos de 4 a 40 KB (uniforme) y 5 000 eventos de llegada o terminación. La carga se controla con el número promedio de procesos vivos: si hay v procesos vivos, uno termina con probabilidad v / (v + objetivo); si no, llega uno nuevo.

Escenario	Procesos vivos (promedio)	Carga ofrecida aprox.
E1 · Carga baja	4	34 %
E2 · Carga media	8	69 %
E3 · Carga alta	13	112 %

Carga ofrecida = procesos vivos × tamaño medio (22 KB) / memoria total. Por encima del 100 % la memoria no alcanza para todos y los rechazos son inevitables.

Diseño del experimento: 3 escenarios × 3 unidades × 10 semillas = 90 secuencias. Cada secuencia se aplica **idéntica** a las tres estrategias, en total 270 ejecuciones. Las 10 semillas permiten reportar media y desviación estándar, y comparar las estrategias de forma pareada (misma secuencia).

Casos fijos: el ejemplo de los apuntes y el ejemplo del quiz, con resultado conocido de antemano.

Métricas (se toma una foto de la memoria después de cada evento y se promedian):

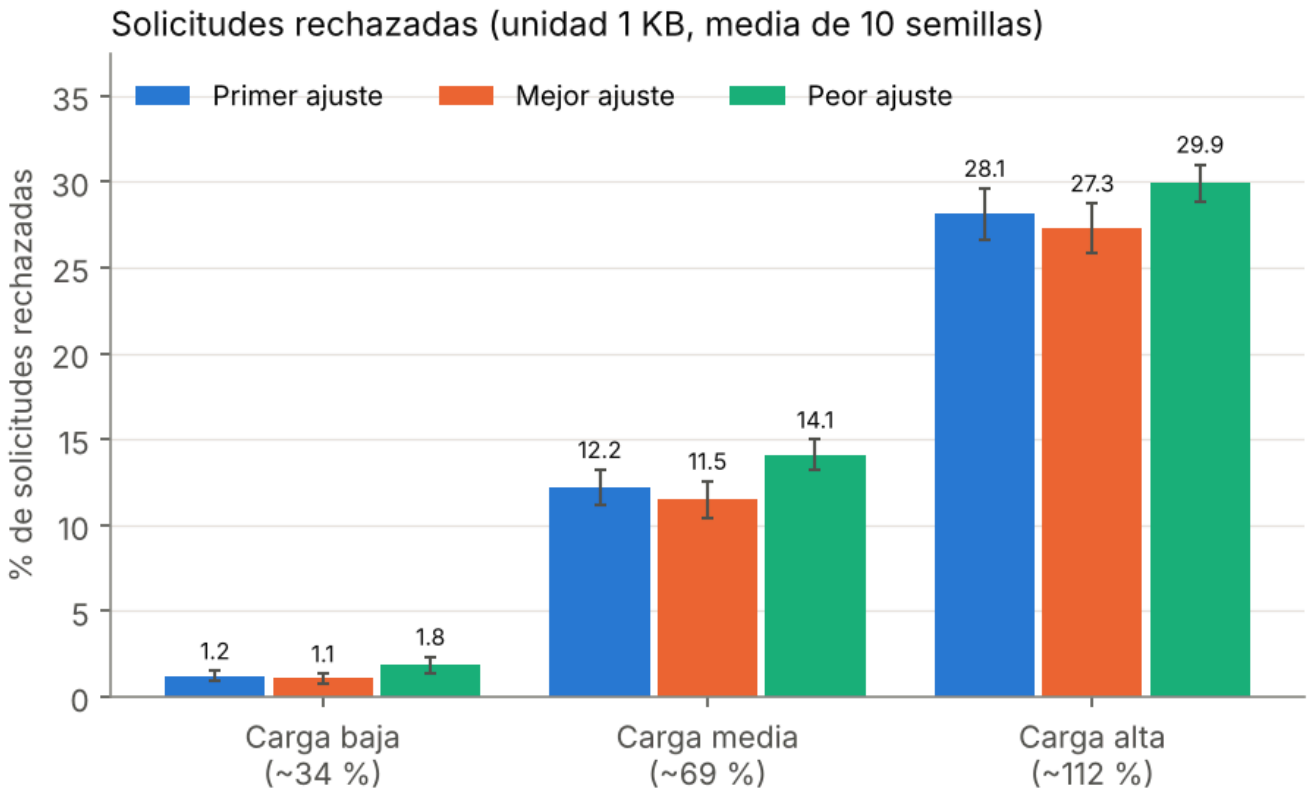
Métrica	Definición	Lectura
Tasa de rechazo	Solicitudes rechazadas / solicitudes válidas	Menor es mejor
Utilización	KB pedidos por los procesos / KB totales	Mayor es mejor; no cuenta el redondeo
Fragmentación externa	1 − hueco mayor / memoria libre total	0 = todo lo libre está junto; cerca de 1 = muy repartido
Fragmentación interna	Suma de (KB asignados − KB pedidos)	KB desperdiciados dentro de particiones
Relación de huecos	Huecos / bloques ocupados	Regla del 50 %: cerca de 0,5
Coste de búsqueda	Bloques revisados por la estrategia, separado en éxitos y rechazos	Menor es más rápido

Los rechazos se clasifican en dos tipos: **por fragmentación**, cuando la memoria libre total sí alcanzaba pero ningún hueco servía, y **sin espacio**, cuando ni siquiera la memoria libre total alcanzaba.

7. Resultados obtenidos

Todos los valores son medias de 10 semillas con unidad de 1 KB, salvo donde se indica; las barras de error marcan una desviación estándar. Los datos completos están en `resultados/resultados.csv` y `resultados/tabla_resumen.csv`.

7.1 Solicitudes rechazadas. El mejor ajuste rechaza menos y el peor ajuste más, en los tres niveles de carga. Con carga media: 11,5 %, 12,2 % y 14,1 %.

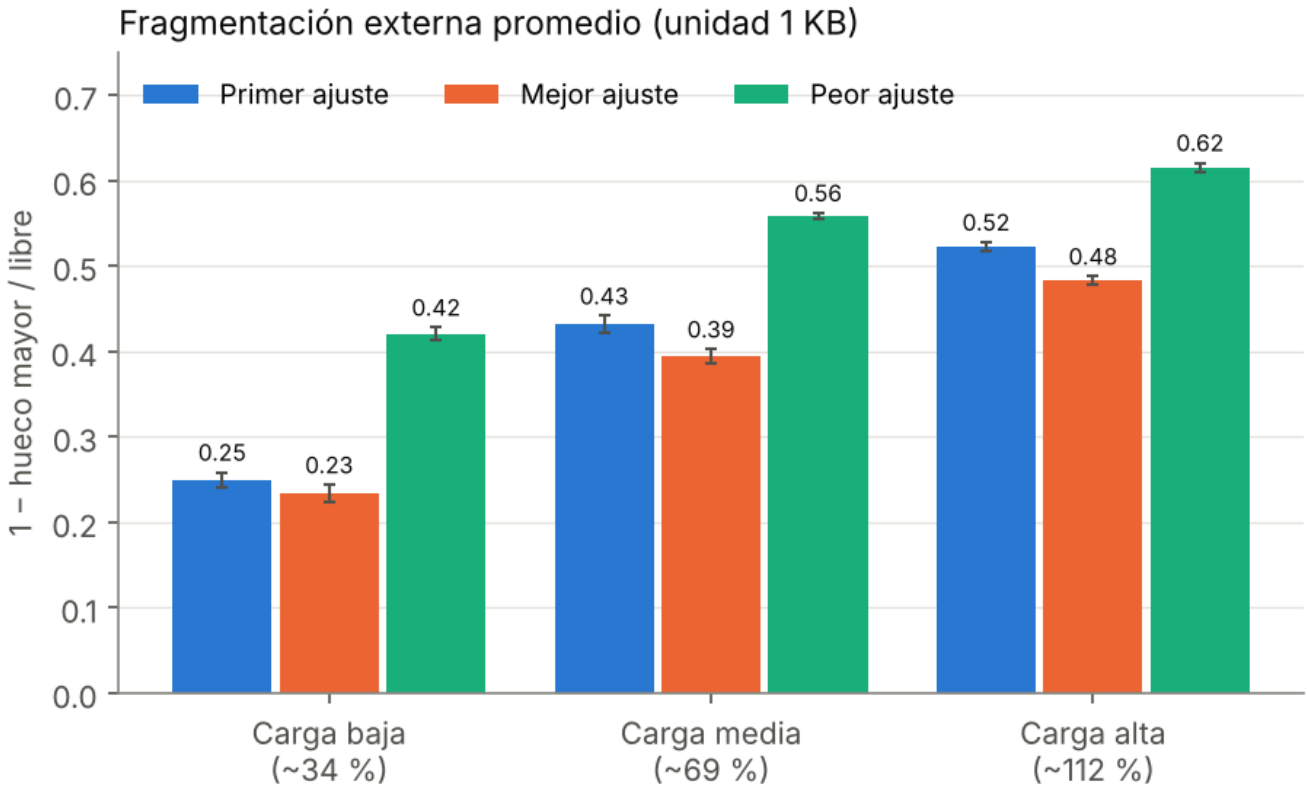


La mayoría de los rechazos se deben a fragmentación externa, no a falta de memoria:

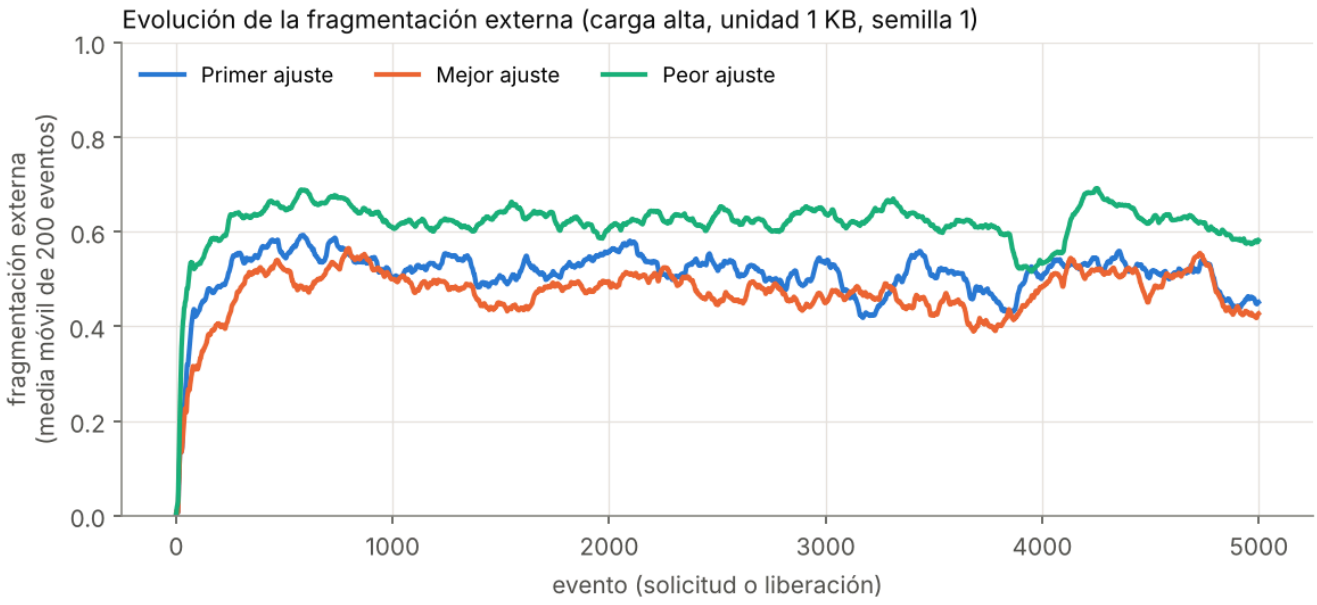
Escenario	Primer ajuste	Mejor ajuste	Peor ajuste
E1 · Carga baja	87 %	84 %	94 %
E2 · Carga media	86 %	80 %	94 %
E3 · Carga alta	85 %	77 %	93 %

Porcentaje de los rechazos en los que había memoria libre suficiente, pero repartida en huecos.

7.2 Fragmentación externa. El peor ajuste deja la memoria libre mucho más repartida en todos los escenarios.



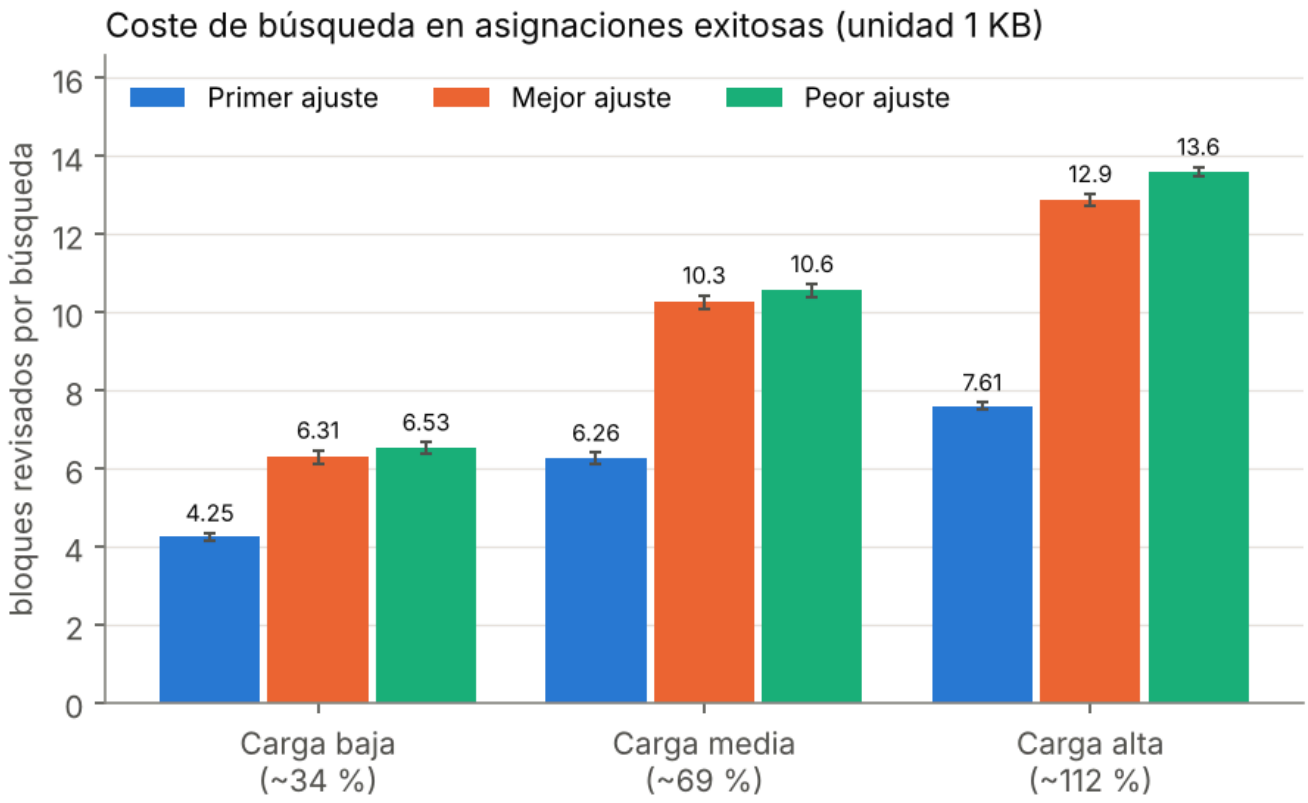
7.3 Evolución en el tiempo. Tras unos 300 eventos de arranque, las diferencias entre estrategias se mantienen estables durante toda la ejecución.



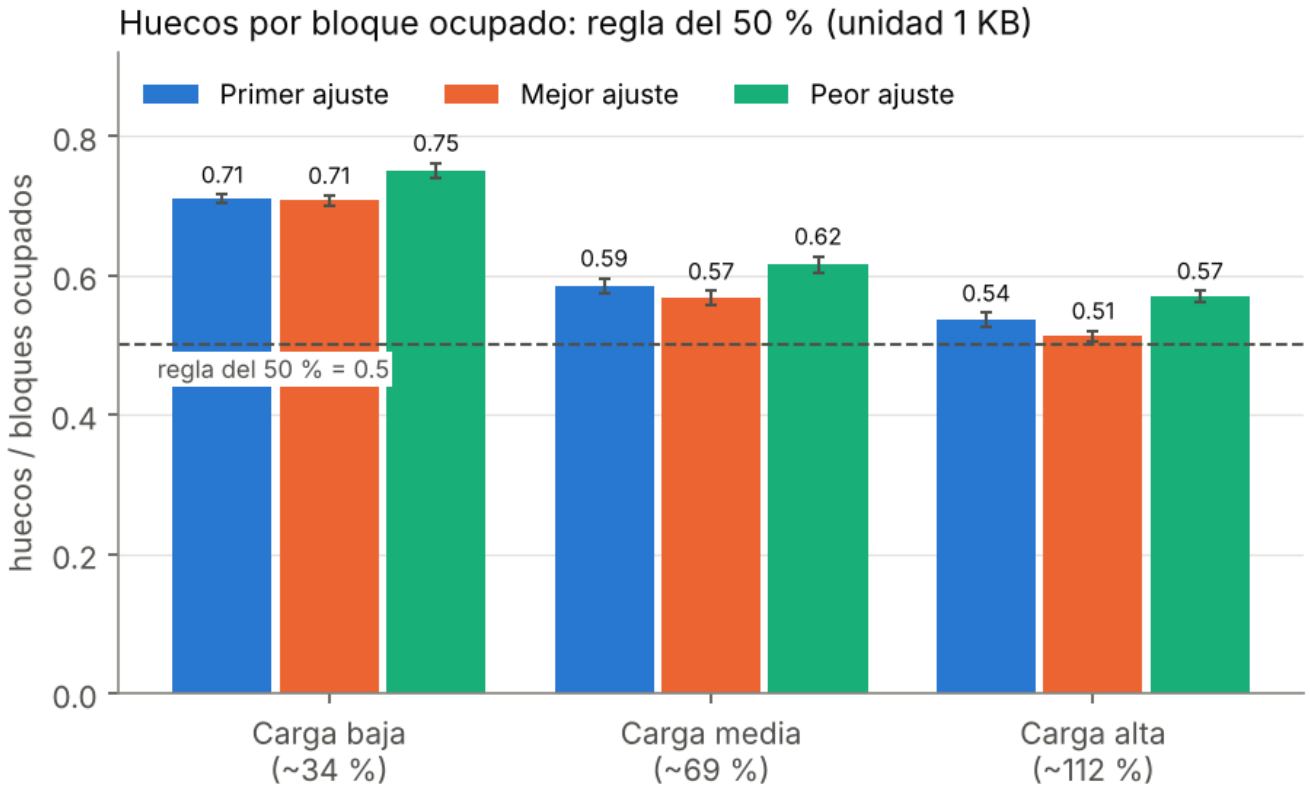
7.4 Utilización de la memoria (KB pedidos / KB totales, promedio en el tiempo):

Escenario	Primer ajuste	Mejor ajuste	Peor ajuste
E1 · Carga baja	37,7 %	37,7 %	37,2 %
E2 · Carga media	59,3 %	60,3 %	56,9 %
E3 · Carga alta	70,1 %	71,9 %	66,7 %

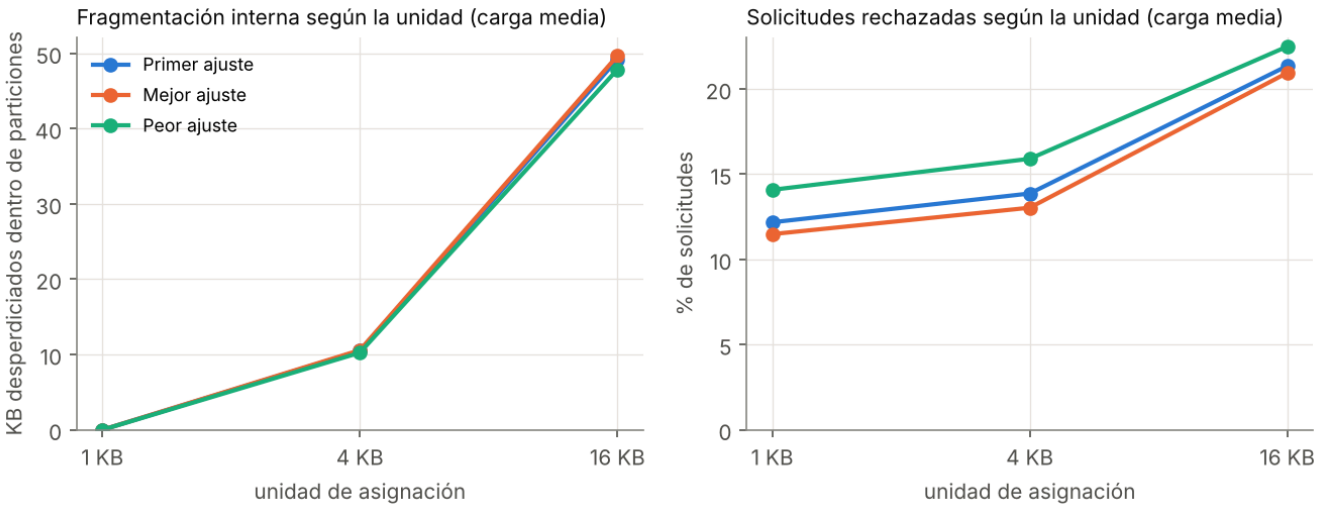
7.5 Coste de búsqueda. En las asignaciones exitosas, el primer ajuste revisa entre un 33 % y un 41 % menos bloques que el mejor ajuste. En los rechazos las tres recorren la lista completa (con carga alta: 15,7, 15,6 y 15,7 bloques).



7.6 Regla del 50 %. La relación huecos / bloques ocupados baja al subir la carga y, con carga alta, el primer ajuste queda en 0,54.



7.7 Efecto de la unidad de asignación (carga media). Al pasar de 1 a 16 KB, la fragmentación interna sube de 0 a unos 49 KB promedio y la tasa de rechazo pasa de 11–14 % a 21–23 %, con las tres estrategias.



7.8 Verificación. Las pruebas automáticas (`make test`) suman **2 921 662 comprobaciones sin fallos**, también bajo AddressSanitizer y UBSan. Incluyen un oráculo que compara cada estrategia con su definición en 90 000 búsquedas y un recálculo

independiente en Python de las métricas en 27 000 estados de memoria, sin diferencias. Se introdujeron 12 errores a propósito en el núcleo y las métricas: las pruebas detectaron los 12 (uno solo después de reforzar la prueba correspondiente).

8. Análisis e interpretación

El peor ajuste es la peor estrategia, de forma consistente. Comparando con la misma secuencia, tuvo más fragmentación externa que el primer ajuste en las 90 comparaciones (9 configuraciones × 10 semillas) y más rechazos en 89 de 90. La causa es que siempre parte el hueco más grande: cuando después llega un proceso grande, ya no queda un hueco que lo contenga. Su idea de "dejar un sobrante útil" no compensa haber destruido los huecos grandes. Coincide con los apuntes: primer y mejor ajuste son superiores al peor ajuste en aprovechamiento del espacio.

Mejor y primer ajuste quedan muy cerca. El mejor ajuste tuvo menos fragmentación externa en las 90 comparaciones, pero en rechazos la ventaja es pequeña: entre 0,7 y 0,9 puntos porcentuales con carga media y alta, del mismo orden que la variación entre semillas (≈ 1 punto), y ganó en 73 de 90 comparaciones. Esto coincide con los apuntes, que no encuentran una diferencia clara de aprovechamiento entre ambas. Al ocupar el hueco más ajustado, el mejor ajuste conserva intactos los huecos grandes; el riesgo teórico de dejar astillas inútiles se nota poco aquí porque los procesos piden al menos 4 KB.

El primer ajuste es el más rápido. Revisa entre un 33 % y un 41 % menos bloques en las búsquedas exitosas, porque se detiene en el primer hueco válido. Un hallazgo metodológico importante: al principio medimos el coste promediando éxitos y rechazos juntos, y las tres estrategias parecían igual de rápidas. La razón es que en un rechazo todas recorren la lista completa. Separar ambos casos cambió la conclusión.

La regla del 50 % se cumple cuando la memoria está llena. Con carga alta, el primer ajuste mantiene 0,54 huecos por bloque ocupado, muy cerca del 0,5 teórico. Con carga baja la relación sube a 0,71 porque la regla supone un sistema en equilibrio con la memoria casi llena; con pocos procesos, cualquier hueco pesa más en la proporción.

La unidad de asignación es un compromiso. Con 16 KB, la fragmentación interna promedio llega a unos 49 KB, cerca del 19 % de la memoria, y los rechazos casi se duplican. A cambio, hay menos huecos y menos repartidos: con carga media, la fragmentación externa del primer ajuste baja de 0,43 a 0,30. Es exactamente el intercambio que describen los apuntes: bloques grandes simplifican la gestión pero desperdician memoria dentro de cada partición.

Conclusión para un sistema operativo. Incluso con la mejor estrategia, entre el 77 % y el 84 % de los rechazos ocurren con memoria libre suficiente. Ninguna estrategia de asignación contigua elimina la fragmentación externa, solo la reduce. Esto explica por qué los sistemas modernos adoptaron la paginación: al permitir un espacio físico no contiguo, la fragmentación externa desaparece y solo queda la interna, acotada a la última página de cada proceso.

9. Riesgos y responsabilidades profesionales

Riesgo	Consecuencia	Cómo se mitigó
Errores en el núcleo que corrompen el mapa de memoria	Métricas falsas presentadas como válidas	Invariantes I1–I7 después de cada operación; el programa se detiene si uno falla
Escritura fuera de los arreglos	Comportamiento indefinido, resultados aleatorios	Límites calculados (2P + 1 bloques), error controlado al llenarse, pruebas con AddressSanitizer y UBSan
Pruebas que pasan aunque el código esté mal	Falsa confianza	Errores introducidos a propósito para comprobar que las pruebas los detectan; una prueba débil se encontró y reforzó
Métricas mal definidas	Conclusiones equivocadas	Coste de búsqueda separado en éxitos y rechazos; los errores de entrada no castigan a la estrategia
Resultados no reproducibles	Nadie puede verificar el informe	Generador propio con semilla fija y misma secuencia para las tres estrategias
Comparaciones injustas	Diferencias atribuidas a la estrategia equivocada	Un solo código de asignación; la estrategia solo elige el hueco

Riesgo	Consecuencia	Cómo se mitigó
Generalizar demasiado	Recomendar una estrategia para cargas que no se probaron	Se declaran las limitaciones de la carga sintética (sección 10)

En un sistema operativo real los riesgos de la gestión de memoria son mayores. Un error al dividir o fundir bloques puede dar a un proceso memoria que pertenece a otro, lo que es un problema de seguridad y no solo de rendimiento. Reutilizar memoria sin limpiarla puede filtrar datos de un proceso a otro. Y una fragmentación alta puede negar servicio a procesos legítimos aunque haya memoria libre.

Responsabilidades del equipo:

- Presentar solo resultados obtenidos de ejecuciones reales, con los datos y el código para reproducirlos.
- Declarar las limitaciones y los hallazgos que contradicen lo esperado, en lugar de ocultarlos.
- Mantener la trazabilidad de los aportes de cada integrante en el repositorio.
- **Uso de inteligencia artificial:** se utilizó un asistente de IA como apoyo en el diseño, la implementación y la redacción. Todo el código fue ejecutado y verificado con pruebas automáticas, y cada integrante revisó los módulos para poder explicarlos en la sustentación.

10. Limitaciones, mejoras y conclusiones

Limitaciones:

- Si ningún hueco sirve, la solicitud se rechaza. Un sistema real podría hacer esperar al proceso o atender antes a otro más pequeño de la cola de entrada, como describen los apuntes.
- La carga es sintética: tamaños uniformes entre 4 y 40 KB y terminaciones al azar. Cargas reales, con muchos procesos pequeños y pocos grandes, podrían cambiar la distancia entre primer y mejor ajuste.
- El tiempo no se simula; solo importa el orden de los eventos. El coste de búsqueda se mide en bloques revisados, no en tiempo de reloj.
- No hay compactación, paginación ni reubicación.

- El generador no conoce el estado de la memoria: cuando un proceso es rechazado, su terminación posterior se ignora, así que con carga alta hay menos procesos vivos que el objetivo.

Mejoras posibles:

1. Añadir *siguiente ajuste* (*next fit*), que retoma la búsqueda donde terminó la anterior.
2. Simular una cola de espera en lugar de rechazar, y medir el tiempo de espera.
3. Añadir compactación y medir cuánto reduce los rechazos frente a su coste en KB movidos.
4. Simular paginación con la misma carga para comparar contra la asignación contigua.
5. Probar distribuciones de tamaño no uniformes.

Conclusiones:

1. El peor ajuste es la peor opción en todas las configuraciones probadas: más rechazos, más fragmentación externa y menor utilización.
2. El mejor ajuste aprovecha la memoria un poco mejor que el primer ajuste, pero la diferencia es pequeña, mientras que el primer ajuste es entre un 33 % y un 41 % más rápido al buscar. Para un sistema real, el primer ajuste es el mejor equilibrio.
3. La regla del 50 % se confirma con la memoria llena: 0,54 huecos por bloque ocupado con primer ajuste.
4. Una unidad de asignación grande reduce la fragmentación externa pero dispara la interna y los rechazos.
5. La fragmentación externa causa la gran mayoría de los rechazos con cualquier estrategia contigua, lo que justifica el paso a la paginación.

11. Referencias utilizadas

1. Universidad de La Laguna, Sistemas Operativos. *Memoria principal* (apuntes de la asignatura). ull-esit-sistemas-operativos.github.io
2. Universidad de La Laguna, Sistemas Operativos. *Paginación* (apuntes de la asignatura). ull-esit-sistemas-operativos.github.io
3. Universidad de La Laguna, Sistemas Operativos. *Memoria virtual* (apuntes de la asignatura). ull-esit-sistemas-operativos.github.io
4. Silberschatz, A., Galvin, P. B. y Gagne, G. (2018). *Operating System Concepts* (10.ª ed.). Wiley. Capítulo 9, Main Memory.

5. Tanenbaum, A. S. y Bos, H. (2014). *Modern Operating Systems* (4.ª ed.). Pearson. Sección 3.2, gestión de memoria con mapas de bits y listas ligadas.
6. Knuth, D. E. (1997). *The Art of Computer Programming, Vol. 1: Fundamental Algorithms* (3.ª ed.). Addison-Wesley. Sección 2.5, Dynamic Storage Allocation (origen de la regla del 50 %).
7. Marsaglia, G. (2003). Xorshift RNGs. *Journal of Statistical Software*, 8(14), 1–6.

Anexo — evidencias en el repositorio: `resultados/resultados.csv` (270 ejecuciones), `resultados/tabla_resumen.csv`, `resultados/comparacion_pareada.csv`, `resultados/traza_ejemplo_apuntes.txt`, `resultados/traza_ejemplo_quiz.txt`, `resultados/resumen_semilla1.txt` y la salida de `make test`. Instrucciones de ejecución en `README.md`.